Enterprise Service Creation Standard

Standardized Blueprint & Sequential Prompts for FastAPI Services (Spring Boot Architecture)

1. Executive Summary & Architectural Vision

This document defines the strict engineering standard for building microservices across teams. By adopting a **Spring Boot inspired N-tier modular architecture** in Python (FastAPI), we enforce separation of concerns, centralized error handling, correlation ID traceability, profile-based configuration, and robust security.

2. Standardized Project Directory Structure

```
service-root/ ├── .env / .env.example # Profile-based env vars (DEV_, UAT_, PROD_) ├── requirements.txt, Dockerfile, docker-compose.yml └── app/ ├── app.py # FastAPI app factory & lifecycle hooks ├── constant.py # Profile constants (dev, uat, prod) & common properties ├── config/ # Database engine & security settings ├── entity/ # SQLAlchemy ORM Entities (JPA equivalent) ├── model/ # Pydantic Request/Response DTOs ├── repo/ # Data Access Layer / Repositories ├── service/ # Business Logic Layer ├── route/ # REST Controllers (health, auth, persons) ├── exception/ # Centralized exception handlers & custom errors ├── logging/ # Correlation ID middleware & structured logging └── common/ # Shared response templates & utilities
```

3. Sequential Development Prompts (Step-by-Step)

Prompt 1: Project Skeleton, Config, and In-Memory DB

Deliverables Establish `.env` profile handling, `constant.py`, `app.py`, entity layer, and H2-style SQLite in-memory database lifecycle with automatic startup seeding.

Prompt 2: Logging, Correlation ID & Centralized Error Handling

Deliverables Implement FastAPI middleware to inject and propagate `correlationId` across all logs/headers. Build custom exceptions and global handlers mapping technical errors to user-friendly messages with standardized error response templates.

Prompt 3: Security, Authentication & Route Protection

Deliverables Implement Bearer token authentication dependencies while exempting public endpoints (`/health`, `/docs`, `/redoc`, `/auth/refresh`). Secure all CRUD resource endpoints.

Prompt 4: Repository, Service & CRUD REST Endpoints

Deliverables Implement repository data access (`app/repo/`), business logic (`app/service/`), Pydantic validation (`app/model/`), and REST endpoints (`app/route/person_route.py`).

Prompt 5: Documentation, Diagrams & Dockerization

Deliverables Create a comprehensive `README.md` containing Index, Tech Stack, HLD/LLD/Sequence diagrams (Mermaid.js), troubleshooting section, `Dockerfile`, and `docker-compose.yml`.

Golden Governance Rules for All Services:

- **Traceability:** Every log entry must include the active `correlationId`.
- **Error Handling:** Never leak raw stack traces to clients; use centralized error response templates.
- **Documentation:** Every service must include live Swagger UI and an interactive Mermaid architecture diagram in its README.